

**LAPORAN PRAKTIKUM
WORKSHOP KECERDASAN BUATAN
BAB 3**



PANDU WINATA KRISNA PUTRA
5324600003

**PROGRAM STUDI SARJANA TERAPAN
TEKNOLOGI REKAYASA MULTIMEDIA
JURUSAN TEKNOLOGI MULTIMEDIA KREATIF
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026**

Example 3.1

```
#Example 3.1 Python SVM Classifications
from sklearn import svm
X = [[170, 70, 10], [180, 80, 12], [170, 65, 8], [160, 55, 7]]
#Height[cm], Weight[kg], Shoesize[UK]
y = [0, 0, 1, 1] #Gender, 0: Male, 1: Female
clf = svm.SVC()
clf.fit(X, y)
#Predict
p = clf.predict([[160, 60, 7]])
print(p)
```

```
(.venv) PS C:\Users\Administrator\Documents\TUGAS\Semester 5\AI> & "c:\Users\Administrator\Documents\TUGAS\Semester 5\AI\AI BAB 2\.venv\Scripts\python.exe" "c:/Users/Administrator/Documents/TUGAS/Semester 5/AI/AI BAB 3/AI_03_01.py"
[1]
```

Example 3.2

```
#Example 3.2 Python SVM Iris Classifications
from sklearn import svm, datasets
iris = datasets.load_iris()
# Take the first two features: Sepal length and Sepal width
X = iris.data[:, :2]
y = iris.target #0: Setosa, 1: Versicolour, 2:Virginica
print(y)
clf = svm.SVC()
clf.fit(X, y)
#Predict the flower for a given Sepal length and width
p = clf.predict([[5.4, 3.2]])
print(p)
```

```
(.venv) PS C:\Users\Administrator\Documents\tugasSemester 5\AI> & "c:\Users\Administrator\Documents\tugasSemester 5\AI\BAB 2\.venv\Scripts\python.exe" "c:/Users/Administrator/Documents/tugasSemester 5/AI/BAB 3/AI_03_02.py"
```

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	2	2	2	2	2
2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	
2	2																						

```
[0]
```

```
(.venv) PS C:\Users\Administrator\Documents\tugasSemester 5\AI>
```

Example 3.3

```
#Example 3.3 Python SVM Iris CSV Classifications
from sklearn import svm, datasets
import pandas as pd
df = pd.read_csv(r'C:\Users\Administrator\Documents\TUGAS\Semester
5\AI\AI BAB 3\iris.csv')
X = df.values[:, :2]
s = df['species']
d = dict([(y,x) for x,y in enumerate(sorted(set(s)))])
y = [d[x] for x in s]
```

```
clf = svm.SVC()
clf.fit(X, y)
#Predict the flower for a given Sepal length and width
p = clf.predict([[5.4, 3.2]])
print(p)
```

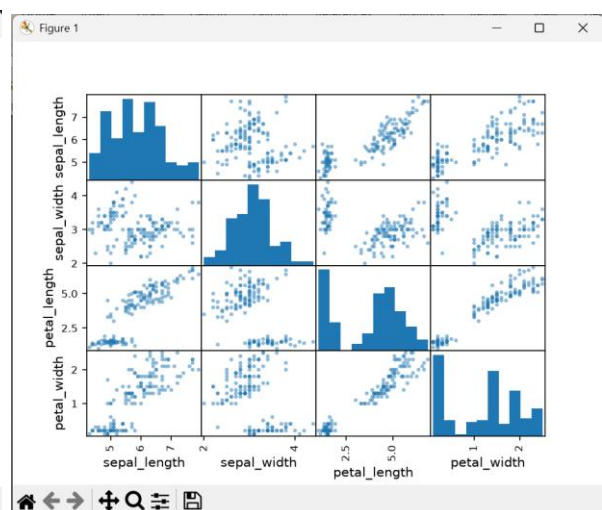
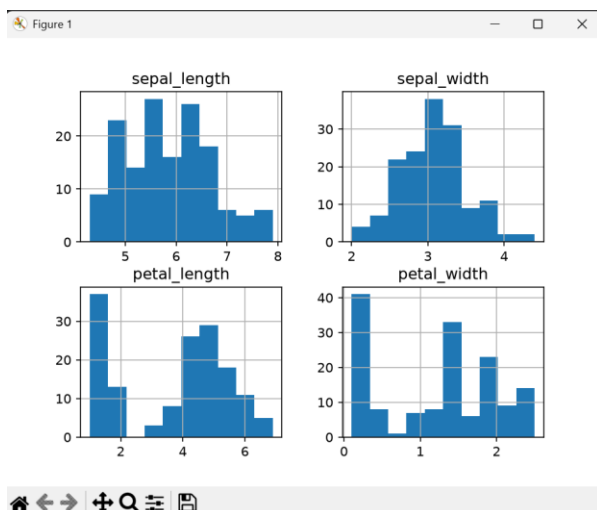
```
• (.venv) PS C:\Users\Administrator\Documents\TUGAS\Semester 5\AI> & "c:\Users\Administrator\Documents\TUGAS\Semester 5\AI\AI BAB 2\.venv\Scripts\python.exe" "c:/Users/Administrator/Documents/TUGAS/Semester 5/AI/AI BAB 3/AI_03_03.py"
[0]
○ (.venv) PS C:\Users\Administrator\Documents\TUGAS\Semester 5\AI> █
```

Example 3.4

```
#Example 3.4 Python SVM Iris URL Classifications
from sklearn import svm, datasets
import pandas as pd
from matplotlib import pyplot
from pandas.plotting import scatter_matrix
df =
pd.read_csv('https://gist.githubusercontent.com/curran/a08a1080b88344b0c8a7/raw/0e7a9b0a5d22642a06d3d5b9bcbad9890c8ee534/iris.csv')
print(df.shape)
print(df.head(10))
print(df.tail(10))
print(df.describe())
#count NAN
print(df.isna().sum().sum())
#drop NAN values
df = df.dropna()
print(df.groupby('species').size())
# histograms
df.hist()
pyplot.show()
# scatter plot matrix
scatter_matrix(df)
pyplot.show()
X = df.values[:, :2]
s = df['species']
d = dict([(y,x) for x,y in enumerate(sorted(set(s)))])
y = [d[x] for x in s]
clf = svm.SVC()
clf.fit(X, y)
#Predict the flower for a given Sepal length and width
p = clf.predict([[5.4, 3.2]])
print(p)
```

```
(150, 5)
  sepal_length  sepal_width  petal_length  petal_width  species
0          5.1          3.5          1.4          0.2    setosa
1          4.9          3.0          1.4          0.2    setosa
2          4.7          3.2          1.3          0.2    setosa
3          4.6          3.1          1.5          0.2    setosa
4          5.0          3.6          1.4          0.2    setosa
5          5.4          3.9          1.7          0.4    setosa
6          4.6          3.4          1.4          0.3    setosa
7          5.0          3.4          1.5          0.2    setosa
8          4.4          2.9          1.4          0.2    setosa
9          4.9          3.1          1.5          0.1    setosa
140         6.7          3.1          5.6          2.4   virginica
141         6.9          3.1          5.1          2.3   virginica
142         5.8          2.7          5.1          1.9   virginica
143         6.8          3.2          5.9          2.3   virginica
144         6.7          3.3          5.7          2.5   virginica
145         6.7          3.0          5.2          2.3   virginica
146         6.3          2.5          5.0          1.9   virginica
147         6.5          3.0          5.2          2.0   virginica
148         6.2          3.4          5.4          2.3   virginica
149         5.9          3.0          5.1          1.8   virginica
```

```
count      150.000000  150.000000  150.000000  150.000000
mean        5.843333   3.054000   3.758667   1.198667
std         0.828066   0.433594   1.764420   0.763161
min         4.300000   2.000000   1.000000   0.100000
25%         5.100000   2.800000   1.600000   0.300000
50%         5.800000   3.000000   4.350000   1.300000
75%         6.400000   3.300000   5.100000   1.800000
max         7.900000   4.400000   6.900000   2.500000
0
species
setosa      50
versicolor  50
virginica   50
dtype: int64
[0]
```



Example 3.5

```
#Example 3.5 Breast Cancer Data
from sklearn.datasets import load_breast_cancer
cancer = load_breast_cancer()
```

```
print(cancer['feature_names'])
print(cancer['data'])
print(cancer.target_names)
```

```
'mean smoothness' 'mean compactness' 'mean concavity'
'mean concave points' 'mean symmetry' 'mean fractal dimension'
'radius error' 'texture error' 'perimeter error' 'area error'
'smoothness error' 'compactness error' 'concavity error'
'concave points error' 'symmetry error' 'fractal dimension error'
'worst radius' 'worst texture' 'worst perimeter' 'worst area'
'worst smoothness' 'worst compactness' 'worst concavity'
'worst concave points' 'worst symmetry' 'worst fractal dimension']
[[1.799e+01 1.038e+01 1.228e+02 ... 2.654e-01 4.601e-01 1.189e-01]
 [2.057e+01 1.777e+01 1.329e+02 ... 1.860e-01 2.750e-01 8.902e-02]
 [1.969e+01 2.125e+01 1.300e+02 ... 2.430e-01 3.613e-01 8.758e-02]
 ...
 [1.660e+01 2.808e+01 1.083e+02 ... 1.418e-01 2.218e-01 7.820e-02]
 [2.060e+01 2.933e+01 1.401e+02 ... 2.650e-01 4.087e-01 1.240e-01]
 [7.760e+00 2.454e+01 4.792e+01 ... 0.000e+00 2.871e-01 7.039e-02]]
['malignant' 'benign']
```

Example 3.6

```
#Example 3.6 Python SVM Breast Cancer Classifications
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
cancer = load_breast_cancer()
X = cancer.data # All of the features
y = cancer.target # All of the labels
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size =
0.2, random_state = 20)
clf = SVC()
clf.fit(X_train, y_train)
#Prediction
y_predict = clf.predict(X_test)
#Print Confusion Matrix and Classification Report
from sklearn.metrics import classification_report,
confusion_matrix
cm = np.array(confusion_matrix(y_test, y_predict, labels=[1,0]))
confusion = pd.DataFrame(cm, index=['is_cancer', 'is_healthy'],
columns=['predicted_cancer', 'predicted_
healthy'])
print(confusion)
print(classification_report(y_test, y_predict))
```

	predicted_cancer	predicted_healthy		
is_cancer	66	0		
is_healthy	8	40		
	precision	recall	f1-score	support
0	1.00	0.83	0.91	48
1	0.89	1.00	0.94	66
accuracy			0.93	114
macro avg			0.95	114
weighted avg			0.94	114

Example 3.7

```
#Example 3.7 Naive Bayes Iris
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
X, y = load_iris(return_X_y=True)
print(X)
clf = GaussianNB()
clf.fit(X, y)
p = clf.predict([[5.0, 3.4, 1.5, 0.4]])
print(p)
```

```
[6.3 3.4 5.6 2.4]
[6.4 3.1 5.5 1.8]
[6.  3.  4.8 1.8]
[6.9 3.1 5.4 2.1]
[6.7 3.1 5.6 2.4]
[6.9 3.1 5.1 2.3]
[5.8 2.7 5.1 1.9]
[6.8 3.2 5.9 2.3]
[6.7 3.3 5.7 2.5]
[6.7 3.  5.2 2.3]
[6.3 2.5 5.  1.9]
[6.5 3.  5.2 2. ]
[6.2 3.4 5.4 2.3]
[5.9 3.  5.1 1.8]]
[0]
```

Example 3.8

```
#Example 3.8 LinearDiscriminantAnalysis Classification
from sklearn.datasets import load_iris
from sklearn.datasets import make_classification
from sklearn.discriminant_analysis import
LinearDiscriminantAnalysis
X, y = make_classification(n_samples=1000, n_features=4,
n_informative=2, n_redundant=0,
random_state=0, shuffle=False)
print(X)
clf = LinearDiscriminantAnalysis()
clf.fit(X, y)
print(clf.predict([[0, 0, 0, 0]]))
```

```

[[ -1.66853167 -1.29901346  0.2746472  -0.60362044]
 [ -2.9728827  -1.08878294  0.70885958  0.42281857]
 [ -0.59614125 -1.37007001 -3.11685659  0.64445203]
 ...
 [  0.91711204  1.10596645  0.86766522 -2.25625012]
 [  0.10027664  1.45875846 -0.44360274 -0.67002328]
 [  1.0415229  -0.01987143  0.15216419 -1.9405334 ]]
[1]

```

Example 3.9

```

#Example 3.9 Principal Component Analysis
from sklearn.datasets import load_iris
from sklearn.datasets import make_classification
from sklearn.decomposition import PCA
X, y = make_classification(n_samples=1000, n_features=4,
n_informative=2, n_redundant=0,
random_state=0, shuffle=False)
print(X)
clf = PCA()
clf.fit(X, y)
print(clf.explained_variance_ratio_)
print(clf.singular_values_)

```

```

[[ -1.66853167 -1.29901346  0.2746472  -0.60362044]
 [ -2.9728827  -1.08878294  0.70885958  0.42281857]
 [ -0.59614125 -1.37007001 -3.11685659  0.64445203]
 ...
 [  0.91711204  1.10596645  0.86766522 -2.25625012]
 [  0.10027664  1.45875846 -0.44360274 -0.67002328]
 [  1.0415229  -0.01987143  0.15216419 -1.9405334 ]]
[0.35054565 0.26955436 0.1920725  0.1878275 ]
[42.31488145 37.10598956 31.32228453 30.97422379]

```

Example 3.10

```

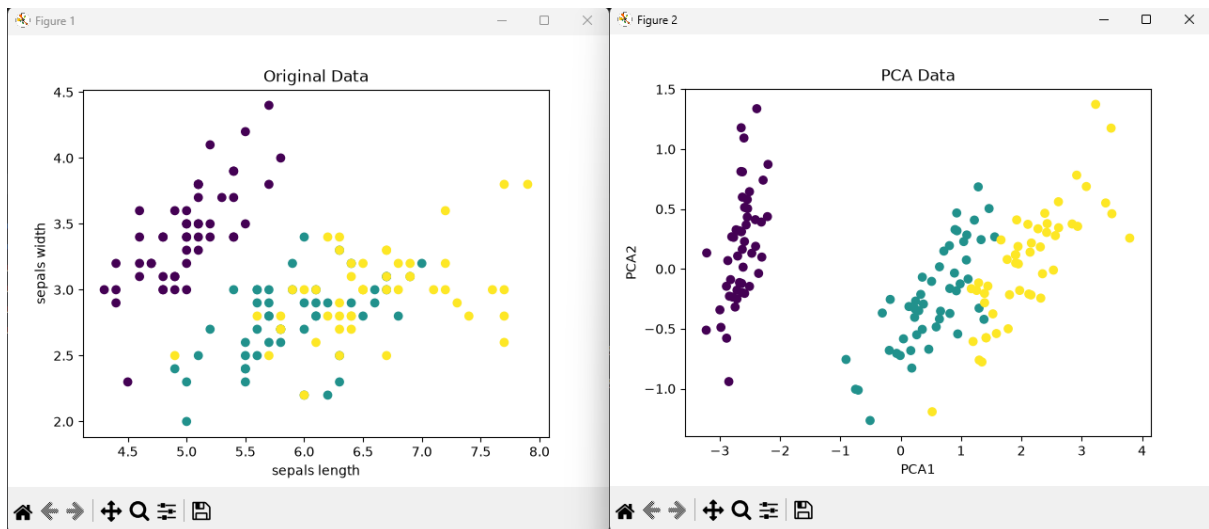
#Example 3.10 Principal Component Analysis Iris
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn import decomposition
from sklearn import datasets
#Load Iris data
iris = datasets.load_iris()
X = iris.data
y = iris.target
#Plot Iris data
f = plt.figure(1)
plt.scatter(X[:,0], X[:,1], c=y)
plt.xlabel('sepals length')
plt.ylabel('sepals width')
plt.title('Original Data')
f.show()
#Perform PCA

```

```

pca = decomposition.PCA(n_components=3)
pca.fit(X)
X1 = pca.transform(X)
#Plot PCA data
g = plt.figure(2)
#y = np.choose(y, [1, 2, 0]).astype(float)
plt.scatter(X1[:, 0], X1[:, 1], c=y)
plt.xlabel('PCA1')
plt.ylabel('PCA2')
plt.title('PCA Data')
g.show()

```



Example 3. 11

```

#Example 3.11 Decision Tree Classification
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.5, random_state=0)
clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
N = y_test.shape[0]
C = (y_test == y_pred).sum()
print("Total points: %d Correctly labeled points : %d" % (N,C))

```

Total points: 75 Correctly labeled points : 72

Example 3.12

```

#Example 3.12 Random Forest
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
X, y = load_iris(return_X_y=True)

```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.5, random_state=0)
clf = RandomForestClassifier()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print("Total points: %d Correctly labeled points : %d" %(y_test.
shape[0],(y_test == y_pred).sum()))
```

Total points: 75 Correctly labeled points : 70

Example 3.13

```
#Example 3.13 K-Nearest Neighbors Classification
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.5, random_state=0)
clf = KNeighborsClassifier()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print("Total points: %d Correctly labeled points : %d" %(y_test.
shape[0],(y_test == y_pred).sum()))
```

Total points: 75 Correctly labeled points : 72

Example 3.14

```
#Example 3.14 Comparison of Different Classification
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.discriminant_analysis import
LinearDiscriminantAnalysis
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.discriminant_analysis import
QuadraticDiscriminantAnalysis
names = [ "SVM", "Naive Bayes", "LDA",
"QDA", "Decision Tree", "Random Forest",
"Nearest Neighbors", "Neural Networks"]
classifiers = [
SVC(),
GaussianNB(),
LinearDiscriminantAnalysis(),
QuadraticDiscriminantAnalysis(),
DecisionTreeClassifier(),
RandomForestClassifier(),
KNeighborsClassifier(),
```

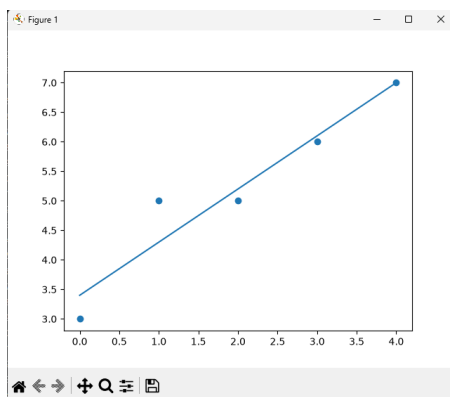
```
MLPClassifier(alpha=1, max_iter=1000)]
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.5, random_state=0)
for name, clf in zip(names, classifiers):
    clf.fit(X_train, y_train)
score = clf.score(X_test, y_test)
print(name + ": " + str(score))
```

Neural Networks: 0.9866666666666667

Example 3.15

```
#Example 3.15 Linear Regression
import matplotlib.pyplot as plt
from scipy import stats
x = [0,1,2,3,4]
y = [3,5,5,6,7]
slope, intercept, r, p, std_err = stats.linregress(x, y)
print("slope: ", slope)
print("intercept: ", intercept)
def myfunc(x):
    return slope * x + intercept
mymodel = list(map(myfunc, x))
plt.scatter(x, y)
plt.plot(x, mymodel)
plt.show()
```

slope: 0.9
intercept: 3.4000000000000004



Example 3.15A

```
#Example 3.15a Linear Regression
import matplotlib.pyplot as plt
import numpy as np
import statsmodels.api as sm
x = [0,1,2,3,4]
y = [3,5,5,6,7]
x1=sm.add_constant(x)
model = sm.OLS(y,x1)
results = model.fit()
print (results.params)
```

```

print (results.summary())
y_pred=results.predict(x1)
plt.scatter(x,y)
plt.xlabel("X")
plt.ylabel("Y")
plt.plot(x,y_pred, "r")
plt.show()

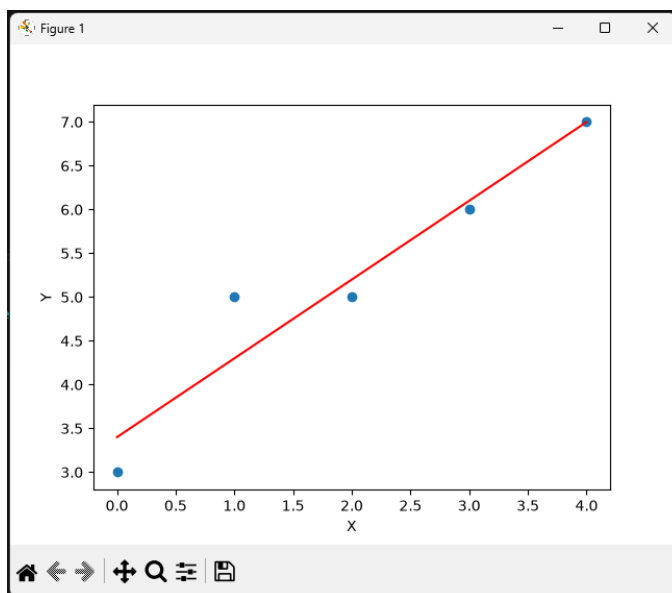
```

```

omni, omnipv = cached("omni_normtest", lambda: omni_normtest(self.wresid))
OLS Regression Results
=====
Dep. Variable:      y      R-squared:      0.920
Model:              OLS      Adj. R-squared:  0.894
Method:             Least Squares      F-statistic:    34.71
Date:               Tue, 15 Sep 2026      Prob (F-statistic): 0.00976
Time:               20:55:20      Log-Likelihood:  -2.1794
No. Observations:   5      AIC:              8.359
Df Residuals:       3      BIC:              7.578
Df Model:           1
Covariance Type:    nonrobust
=====
               coef      std err      t      P>|t|      [0.025      0.975]
-----
const          3.4000      0.374      9.087      0.003      2.209      4.591
x1              0.9000      0.153      5.892      0.010      0.414      1.386
=====
Omnibus:              nan      Durbin-Watson:    2.014
Prob(Omnibus):        nan      Jarque-Bera (JB):  0.901
Skew:                 1.031      Prob(JB):         0.637
Kurtosis:             2.729      Cond. No.         4.74
=====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

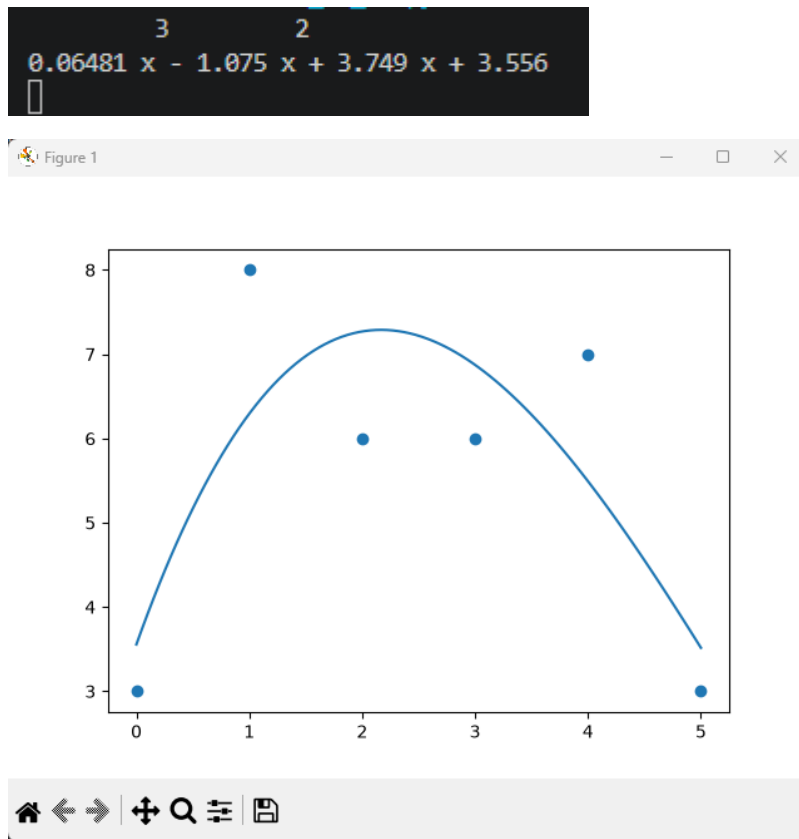


Example 3.16

```

#Example 3.16 Polynomial Regression
import matplotlib.pyplot as plt
from scipy import stats
import numpy as np
x = [0,1,2,3,4,5]
y = [3,8,6,6,7,3]
mymodel = np.poly1d(np.polyfit(x, y, 3))
print(mymodel)
myline = np.linspace(0, 5, 100)
plt.scatter(x, y)
plt.plot(myline, mymodel(myline))
plt.show()

```



Example 3.17

```
#Example 3.17 Least Squares Fitting
import numpy as np
import scipy.optimize as optimization
x = np.array([0,1,2,3,4,5])
y = np.array([100,90,60,30,10,1])
def func(x, a, b, c):
    return a * np.exp(-b * x) + c
popt, pcov = optimization.curve_fit(func, x, y)
print ("Best fit a b c: ",popt)
print ("Best fit covariance: ",pcov)

return a * np.exp(-b * x) + c
Best fit a b c: [ 4.09564521e+02  6.21465806e-02 -3.04106842e+02]
Best fit covariance: [[ 4.92893733e+05 -8.73855445e+01 -4.95831373e+05]
 [-8.73855445e+01  1.55361514e-02  8.79419359e+01]
 [-4.95831373e+05  8.79419359e+01  4.98827106e+05]]
```

Example 3.18

```
#Example 3.18 Multiple Linear Regression
from sklearn import linear_model
import numpy as np
x = np.array([[0,3,5],[1,4,6],[2,5,7],[3,6,8],[4,7,9]])
y = np.array([3,5,5,6,7])
reg = linear_model.LinearRegression()
reg.fit(x, y)
print('Coefficients: \n', reg.coef_)
print('Intercept: \n', reg.intercept_)
```

```
pred = reg.predict([[5,8,10]])
print('Prediction: \n', pred)
```

```
Coefficients:
 [0.3 0.3 0.3]
Intercept:
 0.9999999999999991
Prediction:
 [7.9]
```

Example 3.19

```
#Example 3.19 Logistic Regression
import numpy as np
from sklearn.linear_model import LogisticRegression
X = np.array([[0],[1],[2],[3],[4],[5]])
y = np.array([1,2,3,30,32,31])
clf = LogisticRegression(random_state=0).fit(X, y)
print(clf.predict([[6]]))
print(clf.predict_proba([[6]]))
print(clf.score(X, y))
```

```
[31]
[[0.00111396 0.00810048 0.03285548 0.09767681 0.61033535 0.24991792]]
0.6666666666666666
```

Example 3.20

```
#Example 3.20 K-means Clustering
from sklearn.cluster import KMeans
import numpy as np
X = np.array([[1, 2, 3], [1, 4, 2], [1, 0, 3], \
[10, 2, 4], [9, 4, 3], [11, 0, 2]])
kmeans = KMeans(n_clusters=2, random_state=0).fit(X)
print(kmeans.labels_)
print(kmeans.cluster_centers_)
print(kmeans.predict([[12, 3, 1]]))
```

```
[1 1 1 0 0 0]
[[10.  2.  3.  ]
 [ 1.  2.  2.66666667]]
[0]
```

Example 3.21

```
#Example 3.21 Semi-supervised Learning
import numpy as np
import matplotlib.pyplot as plt
from sklearn.semi_supervised import LabelSpreading
from sklearn.datasets import make_circles
X =
np.array([[0,1],[1,1],[2,0],[3,1],[10,5],[11,6],[12,4],[13,5]])
```

```

y = np.array([0,0,0,0,1,1,1,1])
labels = np.full(8, -1.)
labels[0] = 0
labels[-1] = 1
print(labels)
label_spread = LabelSpreading(kernel='knn', alpha=0.8)
label_spread.fit(X, labels)
output_labels = label_spread.transduction_
print(output_labels)

```

```

[ 0. -1. -1. -1. -1. -1. -1.  1.]
[0. 0. 0. 0. 1. 1. 1. 1.]

```

Example 3.22

```

#Example 3.22 - Q_test.py
#Modified from:
#https://amunategui.github.io/reinforcement-learning/index.html

#http://firsttimeprogrammer.blogspot.com/2016/09/getting-ai-smarter-with-q-learning.html

#http://mnemstudio.org/path-finding-q-learning-tutorial.html
import numpy as np
import pylab as plt
from Q_Utils import *
# Setting up Parameters =====
# Create a routing list and set the goal
points_list = [(0,1),(1,2),(1,3),(2,4),(3,5),(3,6)]
goal = 6
# Show the routing graph
showgraph(points_list)
# The number of points of the R matrix
MATRIX_SIZE = 7
# create matrix R
R = createRmat(MATRIX_SIZE,points_list,goal)
# Create matrix Q
Q = np.matrix(np.zeros([MATRIX_SIZE,MATRIX_SIZE]))
# Learning parameter
gamma = 0.8
# Training =====
scores = []
for i in range(700):
#Select a random current_state (starting point)
    current_state = np.random.randint(0, int(Q.shape[0]))
#Work out all the available next step actions
    available_act = available_actions(R, current_state)
#Choose a random next step action
    action = sample_next_action(available_act)
#Update the Q matrix
    score = update(R,Q,current_state,action,gamma)
    scores.append(score)
    print ('Score:', str(score))
    print("Trained Q matrix:")
    print(Q/np.max(Q)*100)
# Testing =====

```

```

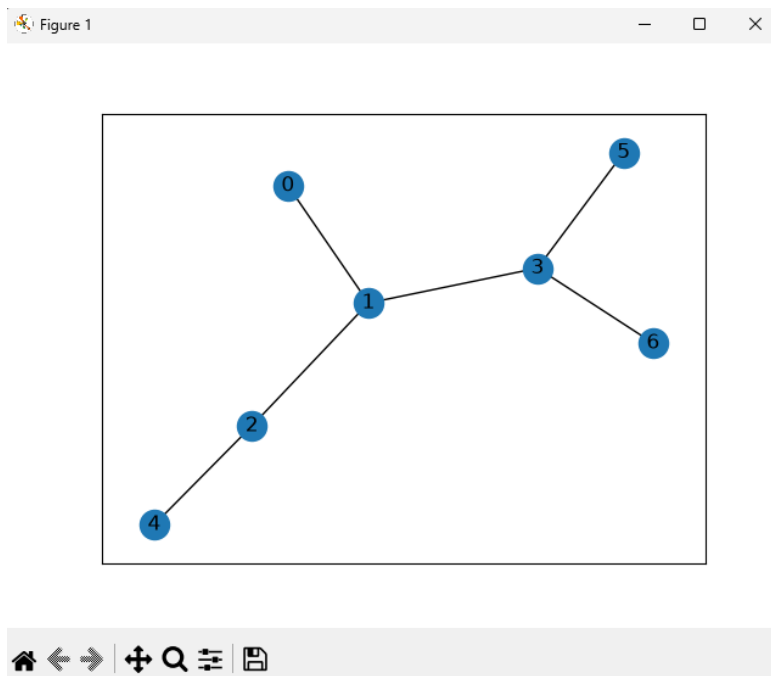
current_state = 0
steps = [current_state]
while current_state != goal:
    next_step_index = np.where(Q[current_state,] ==
np.max(Q[current_state,]))[1]
    if next_step_index.shape[0] > 1:
        next_step_index = int(np.random.choice(next_step_index, size=
1))
    else:
        next_step_index = int(next_step_index)
    steps.append(next_step_index)
    current_state = next_step_index
# Display Results =====
print("Most efficient path:")
print(steps)
plt.plot(scores)
plt.show()

```

```

(0, 1)
(1, 2)
(1, 3)
(2, 4)
(3, 5)
(3, 6)
[[ -1.  0. -1. -1. -1. -1. -1.]
 [ 0. -1.  0.  0. -1. -1. -1.]
 [-1.  0. -1. -1.  0. -1. -1.]
 [-1.  0. -1. -1. -1.  0. 100.]
 [-1. -1.  0. -1. -1. -1. -1.]
 [-1. -1. -1.  0. -1. -1. -1.]
 [-1. -1. -1.  0. -1. -1. 100.]]
Traceback (most recent call last):

```



Example 3.23

```

# Example 3.23 OpenAI Gym CartPole
#https://gym.openai.com/docs/
#https://gym.openai.com/envs/CartPole-v0/
import gym

```

```

env = gym.make('CartPole-v0')
for i_episode in range(20):
    observation = env.reset()
    for t in range(100):
        env.render()
        print(observation)
        action = env.action_space.sample()
        observation, reward, done, info = env.step(action)
        if done:
            print("Episode finished after {} timesteps".format(t+1))
            break
    env.close()

```

Example 3.24

```

# Example 3.24 Ensemble1.py
from sklearn import datasets
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.svm import SVC
iris = datasets.load_iris()
X, y = iris.data[:, 1:3], iris.target
clf1 = LogisticRegression(random_state=1)
clf2 = RandomForestClassifier(n_estimators=50, random_state=1)
clf3 = GaussianNB()
clf4 = SVC()
ecf = VotingClassifier(
    estimators=[('lr', clf1), ('rf', clf2), ('gnb', clf3), ('svc',
    clf4)],
    voting='hard')
for clf, label in zip([clf1, clf2, clf3, clf4, ecf],
    ['LogisticRegression', 'Random Forest', 'naive Bayes', 'SVM',
    'Ensemble']):
    scores = cross_val_score(clf, X, y,
    scoring='accuracy', cv=5)
    print("Accuracy: %0.2f (+/- %0.2f) [%s]" % (scores.mean(),
    scores.std(), label))

```

```

Accuracy: 0.95 (+/- 0.04) [Ensemble]

```

Example 3.25

```

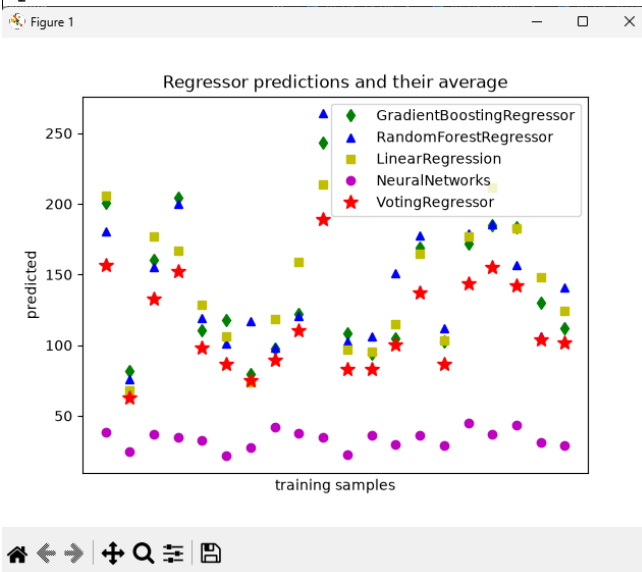
# Example 3.25 Ensemble2.py
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import VotingRegressor
from sklearn.neural_network import MLPRegressor
X, y = load_diabetes(return_X_y=True)
# Train classifiers
reg1 = GradientBoostingRegressor(random_state=1)

```

```

reg2 = RandomForestRegressor(random_state=1)
reg3 = LinearRegression()
reg4 = MLPRegressor()
reg1.fit(X, y)
reg2.fit(X, y)
reg3.fit(X, y)
reg4.fit(X, y)
ereg = VotingRegressor(estimators=[('gb', reg1), ('rf', reg2),
('lr',
reg3), ('NN', reg4)])
print(ereg.fit(X, y))
# %%
# Making predictions
# -----
#
# Now we will use each of the regressors to make the 20 first
predictions.
xt = X[:20]
pred1 = reg1.predict(xt)
pred2 = reg2.predict(xt)
pred3 = reg3.predict(xt)
pred4 = reg4.predict(xt)
pred5 = ereg.predict(xt)
plt.figure()
plt.plot(pred1, 'gd', label='GradientBoostingRegressor')
plt.plot(pred2, 'b^', label='RandomForestRegressor')
plt.plot(pred3, 'ys', label='LinearRegression')
plt.plot(pred4, 'mo', label='NeuralNetworks')
plt.plot(pred5, 'r*', ms=10, label='VotingRegressor')
plt.tick_params(axis='x', which='both', bottom=False, top=False,
labelbottom=False)
plt.ylabel('predicted')
plt.xlabel('training samples')
plt.legend(loc="best")
plt.title('Regressor predictions and their average')
plt.show()

```



Example 3.26

```
# Example 3.26 Auto_ml_test.py
from auto_ml import Predictor
from auto_ml.utils import get_boston_dataset
df_train, df_test = get_boston_dataset()
column_descriptions = {
    'MEDV': 'output',
    'CHAS': 'categorical'
}
ml_predictor =
Predictor(type_of_estimator='regressor', column_descriptions=column
_descriptions)
ml_predictor.train(df_train)
ml_predictor.score(df_test, df_test.MEDV)
```

Example 3.27

```
#Example 3.27 The PyCaret_demo.py
#!pip install pycaret
import pandas as pd
from sklearn import datasets
iris = datasets.load_iris(as_frame=True)
iris.data['Target'] = iris.target
iris = iris.data
iris.head()
from pycaret import classification
classification.setup(data= iris, target='Target')
classification.compare_models()
```

Example 3.28A

```
!pip install lazypredict

*** Requirement already satisfied: lazypredict in /usr/local/lib/python3.13/dist-packages (0.3.0)
Requirement already satisfied: click in /usr/local/lib/python3.13/dist-packages (from lazypredict) (8.5.0)
Requirement already satisfied: scikit-learn>=1.0 in /usr/local/lib/python3.13/dist-packages (from lazypredict) (1.6.1)
Requirement already satisfied: pandas>=1.3 in /usr/local/lib/python3.13/dist-packages (from lazypredict) (2.2.3)
Requirement already satisfied: tqdm>=4.0 in /usr/local/lib/python3.13/dist-packages (from lazypredict) (4.67.3)
Requirement already satisfied: joblib>=1.0 in /usr/local/lib/python3.13/dist-packages (from lazypredict) (1.6.0)
Requirement already satisfied: numpy>=1.21 in /usr/local/lib/python3.13/dist-packages (from lazypredict) (2.1.3)
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist-packages (from joblib>=1.0->lazypredict) (3.1.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages (from pandas>=1.3->lazypredict) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas>=1.3->lazypredict) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas>=1.3->lazypredict) (2026.3)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn>=1.0->lazypredict) (1.16.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn>=1.0->lazypredict) (3.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.8.2->pandas>=1.3->lazypredict) (1.17.0)
```

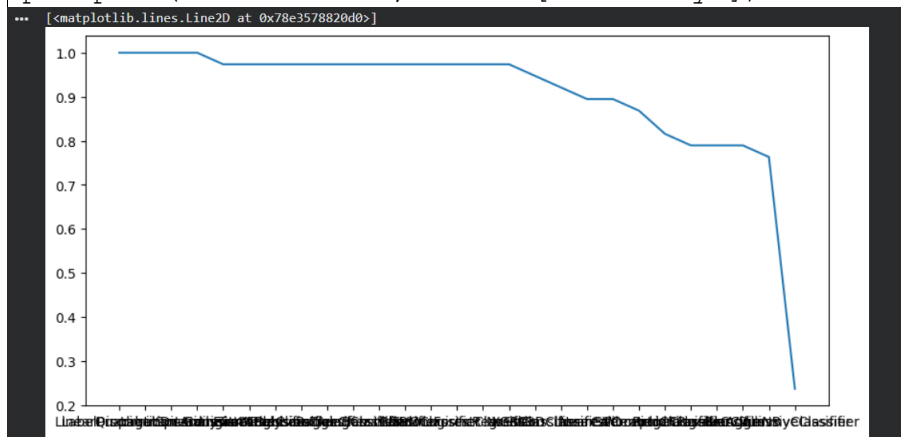
Example 3.28B

```
import lazypredict
from lazypredict.Supervised import LazyClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X,
y, test_size=.25, random_state =1)
clf = LazyClassifier()
models, predictions = clf.fit(X_train, X_test, y_train, y_test)
print(models)
```

Model			
LabelPropagation	1.000000	1.000000	1.000000
LinearDiscriminantAnalysis	1.000000	1.000000	1.000000
LabelSpreading	1.000000	1.000000	1.000000
QuadraticDiscriminantAnalysis	1.000000	1.000000	1.000000
GaussianNB	0.973684	0.979167	1.000000
ExtraTreesClassifier	0.973684	0.979167	1.000000
ExtraTreeClassifier	0.973684	0.979167	0.982759
KNeighborsClassifier	0.973684	0.979167	0.998948
DecisionTreeClassifier	0.973684	0.979167	0.982759
BaggingClassifier	0.973684	0.979167	1.000000
AdaBoostClassifier	0.973684	0.979167	0.973932
SVC	0.973684	0.979167	NaN
LGBMClassifier	0.973684	0.979167	0.984986
RandomForestClassifier	0.973684	0.979167	1.000000
LogisticRegression	0.973684	0.979167	1.000000
NuSVC	0.973684	0.979167	NaN
XGBClassifier	0.947368	0.958333	0.971972
SGDClassifier	0.921053	0.937500	NaN

Example 3.28C

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 5))
plt.plot(models.index, models['Accuracy'])
```



Example 3.28D

```
import lazypredict
from lazypredict.Supervised import LazyRegressor
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.utils import shuffle
import numpy as np
from sklearn.datasets import fetch_california_housing

X, y = fetch_california_housing(return_X_y=True, as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=.1, random_state=1)

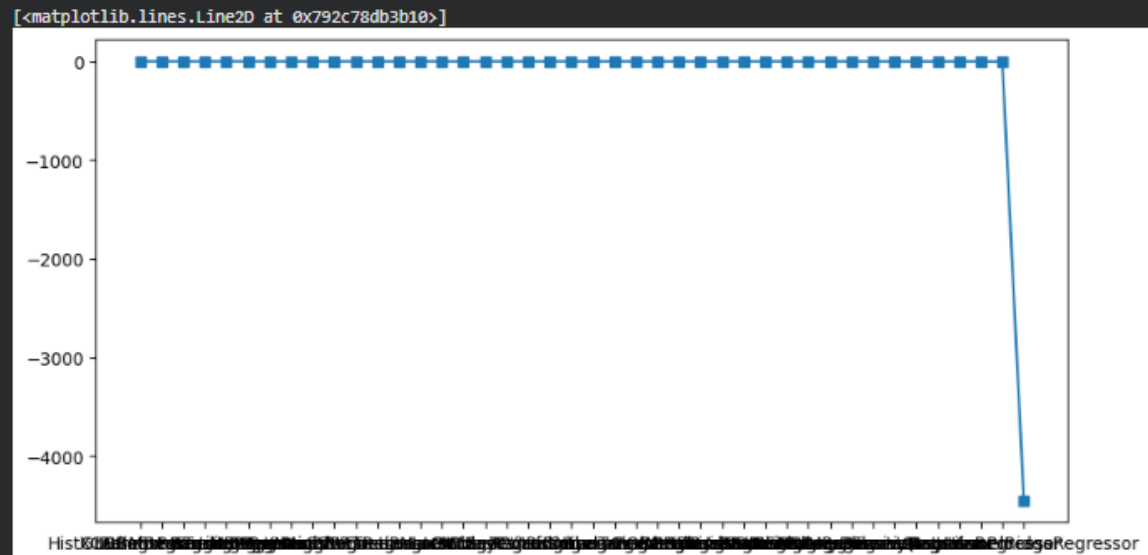
reg = LazyRegressor()
models, predictions = reg.fit(X_train, X_test, y_train, y_test)
print(models)
```

...	PoissonRegressor	0.446211	0.448358	0.861285
	AdaBoostRegressor	0.442738	0.444899	0.863982
	RANSACRegressor	0.426536	0.428760	0.876451
	TweedieRegressor	0.399004	0.397750	0.900725
	GammaRegressor	0.381123	0.383523	0.910492
	PassiveAggressiveRegressor	0.264452	0.267304	0.992614
	ElasticNet	0.200488	0.203588	1.034873
	DummyRegressor	-0.004788	-0.000891	1.160144
	LassoLars	-0.004788	-0.000891	1.160144
	Lasso	-0.004788	-0.000891	1.160144
	QuantileRegressor	-0.045968	-0.041911	1.183579
	KernelRidge	-2.613568	-2.599555	2.200102
	GaussianProcessRegressor	-4467.893034	-4450.563347	77.370368

	Time Taken
Model	0.314153
XGBRegressor	0.289217
LogitRegressor	0.336540
HistGradientBoostingRegressor	15.979664
RandomForestRegressor	6.209644
ExtraTreesRegressor	3.209080
BaggingRegressor	5.286977
MLPRegressor	5.099827
GradientBoostingRegressor	14.385255
NaiveBayes	10.480315
KNeighborsRegressor	0.205668
DecisionTreeRegressor	0.548065
ElasticNetCV	0.589583
LassoCV	0.466587
LarsCV	0.002009
LassoLarsCV	0.089710
RidgeCV	0.034372
HuberRegressor	0.124887
BayesianRidge	0.062663
Ridge	0.014235
LassoLarsIC	0.072969
Lars	0.026915
TransformedTargetRegressor	0.014611
LinearRegression	0.072989
LinearSVR	1.044161
OrthogonalMatchingPursuitCV	0.025860
SGDRegressor	0.059251
ExtraTreeRegressor	0.111479
OrthogonalMatchingPursuit	0.015024
PoissonRegressor	0.034552
AdaBoostRegressor	2.027567
RANSACRegressor	0.113351
TweedieRegressor	0.018182
GammaRegressor	0.039701
PassiveAggressiveRegressor	0.037956
ElasticNet	0.108954
DummyRegressor	0.036189
LassoLars	0.040833
Lasso	0.059733

Example 3.28E

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 5))
plt.plot(models.index, models['R-Squared'], '-s')
```



Exercise 3.1

```
from sklearn import svm

# 6 samples of [Height(cm), Weight(kg), ShoeSize(UK)]
X = [
    [170, 70, 10],
    [180, 80, 12],
```

```

    [170, 65, 8],
    [160, 55, 7],
    [175, 75, 9],
    [155, 50, 6]
]

# 6 target labels: 0 for Male, 1 for Female
y = [0, 0, 1, 1, 0, 1]

clf = svm.SVC()
clf.fit(X, y)

# Predict for a test input
p = clf.predict([[160, 60, 7]])
print("Prediction:", p)

```

```

(.venv) PS C:\Users\Administrator\Documents\TUGAS\Semester 5\AI> & "c:\Users\Administrator\Documents\TUGAS\Semester 5\AI\AI BAB 2\.venv\Scripts\python.exe" "c:/Users/Administrator/Documents/TUGAS/Semester 5/AI/AI BAB 3/AI_03_01-01.py"
Prediction: [1]

```

Exercise 3.2

```

from sklearn import svm, datasets

iris = datasets.load_iris()

# Take 3rd and 4th features: Petal length (col index 2) and Petal
width (col index 3)
X = iris.data[:, 2:4]
y = iris.target # 0: Setosa, 1: Versicolor, 2: Virginica

clf = svm.SVC()
clf.fit(X, y)

# Predict using petal length and width (e.g., 1.5 cm length, 0.3
cm width)
p = clf.predict([[1.5, 0.3]])
print("Prediction:", p)

```

```
Prediction: [0]
```

Exercise 3.3

```

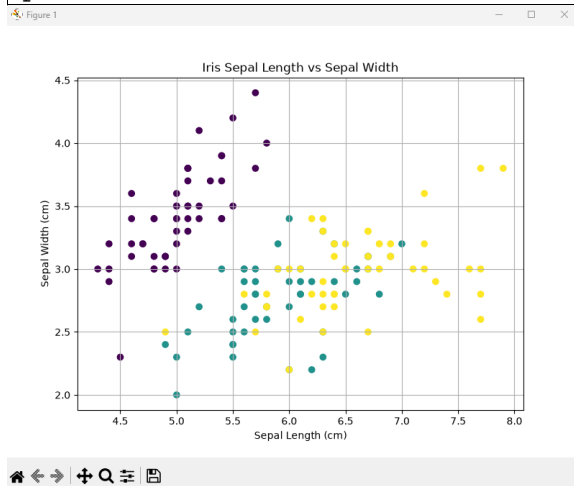
import pandas as pd
from matplotlib import pyplot as plt

url =
'https://gist.githubusercontent.com/curran/a08a1080b88344b0c8a7/ra
w/0e7a9b0a5d22642a06d3d5b9bcbad9890c8ee534/iris.csv'
df = pd.read_csv(url)

# Encode species names into numerical values for coloring points
species_map = {species: i for i, species in
enumerate(df['species'].unique())}
colors = df['species'].map(species_map)

```

```
# Scatter plot of Sepal Length vs Sepal Width
plt.figure(figsize=(8, 6))
plt.scatter(df['sepal_length'], df['sepal_width'], c=colors,
            cmap='viridis')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.title('Iris Sepal Length vs Sepal Width')
plt.grid(True)
plt.show()
```



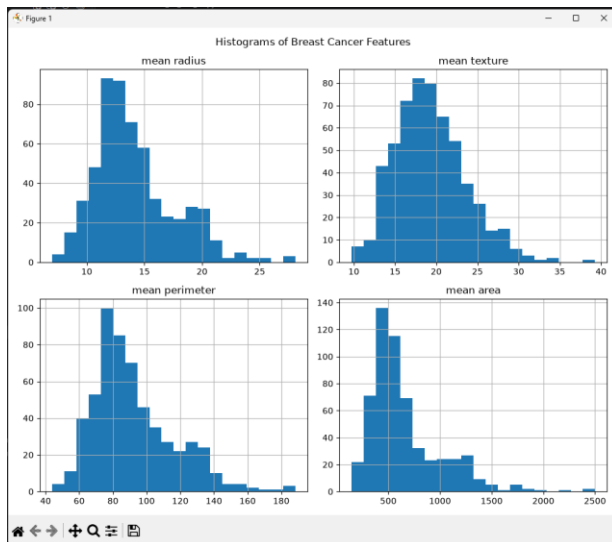
Exercise 3.4

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer

cancer = load_breast_cancer()
df = pd.DataFrame(cancer.data, columns=cancer.feature_names)

# Select four specified feature columns
selected_features = ['mean radius', 'mean texture', 'mean
perimeter', 'mean area']
df[selected_features].hist(bins=20, figsize=(10, 8))

plt.suptitle('Histograms of Breast Cancer Features')
plt.tight_layout()
plt.show()
```



Exercise 3.5

```
from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB
import joblib

# Load data
X, y = load_iris(return_X_y=True)

# Train model
clf = GaussianNB()
clf.fit(X, y)

# Serialize (Save) the model
joblib.dump(clf, 'iris_nb_model.pkl')
print("Model successfully saved to 'iris_nb_model.pkl'.")

# Deserialize (Load) the model back
clf_loaded = joblib.load('iris_nb_model.pkl')
print("Model successfully loaded.")

# Make prediction with the loaded model
p = clf_loaded.predict([[5.0, 3.4, 1.5, 0.4]])
print("Prediction from loaded model:", p)
```

```
Model successfully saved to 'iris_nb_model.pkl'.
Model successfully loaded.
Prediction from loaded model: [0]
```

Exercise 3.6

```
from sklearn.datasets import make_classification
from sklearn.discriminant_analysis import
LinearDiscriminantAnalysis

# Generate dataset with 2000 samples and 6 features
X, y = make_classification(
    n_samples=2000,
    n_features=6,
```

```

        n_informative=3,
        n_redundant=0,
        random_state=0,
        shuffle=False
    )

    clf = LinearDiscriminantAnalysis()
    clf.fit(X, y)

    # Predict sample with 6 input feature values
    p = clf.predict([[0, 0, 0, 0, 0, 0]])
    print("Prediction:", p)

```

Prediction: [0]

Exercise 3.7

```

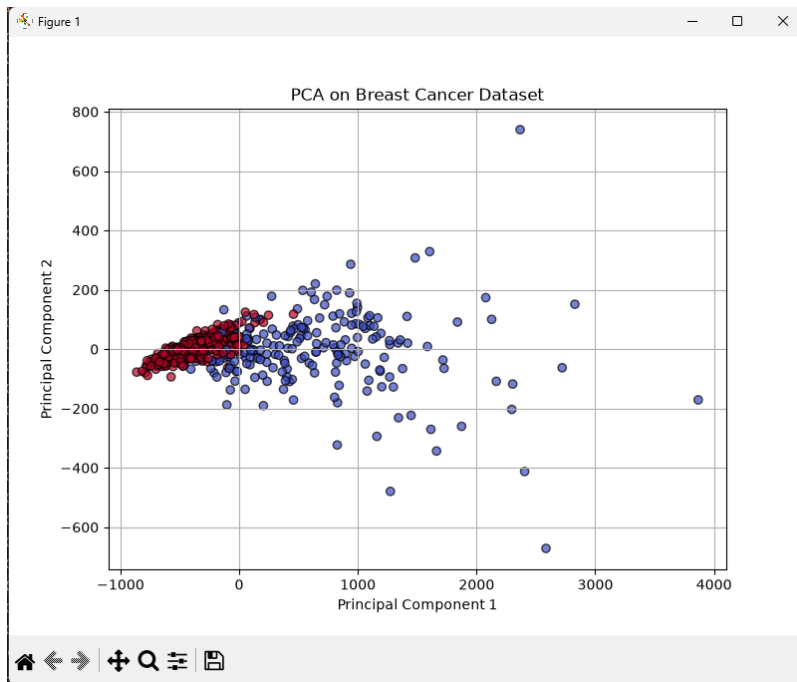
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.decomposition import PCA

cancer = load_breast_cancer()
X = cancer.data
y = cancer.target # 0: Malignant, 1: Benign

# Apply PCA to reduce 30 features down to 2 principal components
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# Plot transformed data
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='coolwarm',
            edgecolor='k', alpha=0.7)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA on Breast Cancer Dataset')
plt.grid(True)
plt.show()

```



Exercise 3.8

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Load wine dataset
wine = load_wine()
X, y = wine.data, wine.target

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.5, random_state=0)

clf = DecisionTreeClassifier(random_state=0)
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

N = y_test.shape[0]
C = (y_test == y_pred).sum()
print(f"Total points: {N} | Correctly labeled points: {C} |
Accuracy: {C/N:.2%}")
```

Total points: 89 | Correctly labeled points: 77 | Accuracy: 86.52%

Exercise 3.9

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Load diabetes dataset
diabetes = load_diabetes()
X, y = diabetes.data, diabetes.target
```

```

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=0)

# Use RandomForestRegressor for continuous targets
rf_regressor = RandomForestRegressor(n_estimators=100,
random_state=0)
rf_regressor.fit(X_train, y_train)

y_pred = rf_regressor.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"R2 Score: {r2:.2f}")

```

```

Mean Squared Error (MSE): 3523.62
R2 Score: 0.31

```

Exercise 3.10

```

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.discriminant_analysis import
LinearDiscriminantAnalysis, QuadraticDiscriminantAnalysis
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier

# Model names
names = [
    "SVM", "Naive Bayes", "LDA", "QDA",
    "Decision Tree", "Random Forest", "Nearest Neighbors", "Neural
Networks"
]

# Classifier instances
classifiers = [
    SVC(),
    GaussianNB(),
    LinearDiscriminantAnalysis(),
    QuadraticDiscriminantAnalysis(),
    DecisionTreeClassifier(),
    RandomForestClassifier(),
    KNeighborsClassifier(),
    MLPClassifier(alpha=1, max_iter=1000)
]

# Load Diabetes dataset
X, y = load_diabetes(return_X_y=True)

```

```

# Note: The target 'y' in load_diabetes is continuous (a
regression task).
# To use classifiers, we convert target 'y' into binary classes
(above/below median).
y_binary = (y > y.mean()).astype(int)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y_binary, test_size=0.5, random_state=0
)

# Train and evaluate models
for name, clf in zip(names, classifiers):
    clf.fit(X_train, y_train)
    score = clf.score(X_test, y_test)
    print(name + ": " + str(score))

```

```

SVM: 0.746606334841629
Naive Bayes: 0.6923076923076923
LDA: 0.7285067873303167

```

Exercise 3.11

```

import matplotlib.pyplot as plt
from scipy import stats

# Expanded dataset (more points)
x = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
y = [3, 5, 5, 6, 7, 8, 9, 11, 12, 14]

# Perform linear regression
slope, intercept, r, p, std_err = stats.linregress(x, y)
print("slope: ", slope)
print("intercept: ", intercept)

# Linear function definition
def myfunc(x_val):
    return slope * x_val + intercept

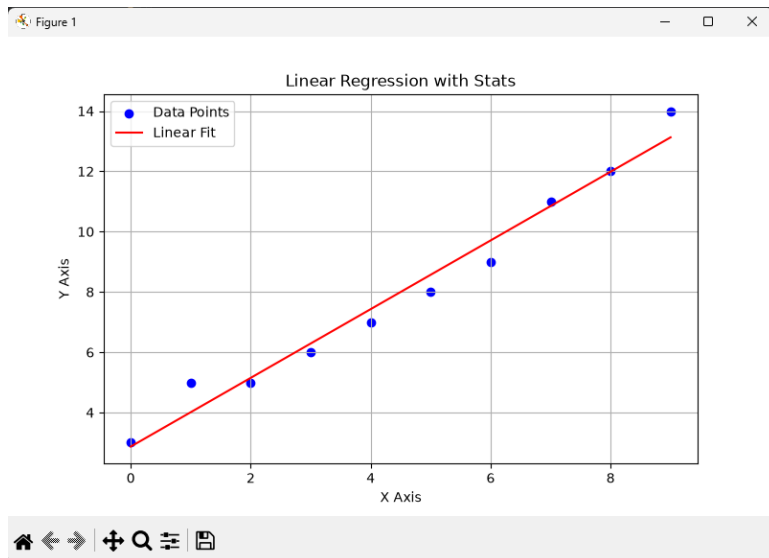
# Map predictions
mymodel = list(map(myfunc, x))

# Plotting with enhancements
plt.figure(figsize=(8, 5))
plt.scatter(x, y, color='blue', label='Data Points')
plt.plot(x, mymodel, color='red', label='Linear Fit')

# Add Labels, Title, Legend, and Grid
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.title("Linear Regression with Stats")
plt.legend()
plt.grid(True)

plt.show()

```



Exercise 3.12

```
from sklearn.datasets import load_linnerud
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# 1. Load the Linnerud multi-output dataset
data = load_linnerud()
X, y = data.data, data.target

# 2. Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# 3. Fit multi-output Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# 4. Make predictions and evaluate
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)

print("Linnerud Feature Names:", data.feature_names)
print("Linnerud Target Names:", data.target_names)
print("Mean Squared Error (MSE):", round(mse, 4))
```

```
Linnerud Feature Names: ['Chins', 'Situps', 'Jumps']
Linnerud Target Names: ['Weight', 'Waist', 'Pulse']
Mean Squared Error (MSE): 242.1476
```

Exercise 3.13

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# 1. Generate 2D synthetic blob dataset
```

```

X, y_true = make_blobs(
    n_samples=300, centers=4, cluster_std=0.60, random_state=0
)

# 2. Fit K-Means with k=4
kmeans = KMeans(n_clusters=4, random_state=0)
y_kmeans = kmeans.fit_predict(X)

# 3. Plot cluster assignments and cluster centroids
plt.figure(figsize=(8, 5))
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis',
            label='Data Points')

# Mark centroids
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200,
            alpha=0.75, marker='X', label='Centroids')

plt.title("K-Means Clustering (make_blobs)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.grid(True)
plt.show()

```



Exercise 3.14

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVR

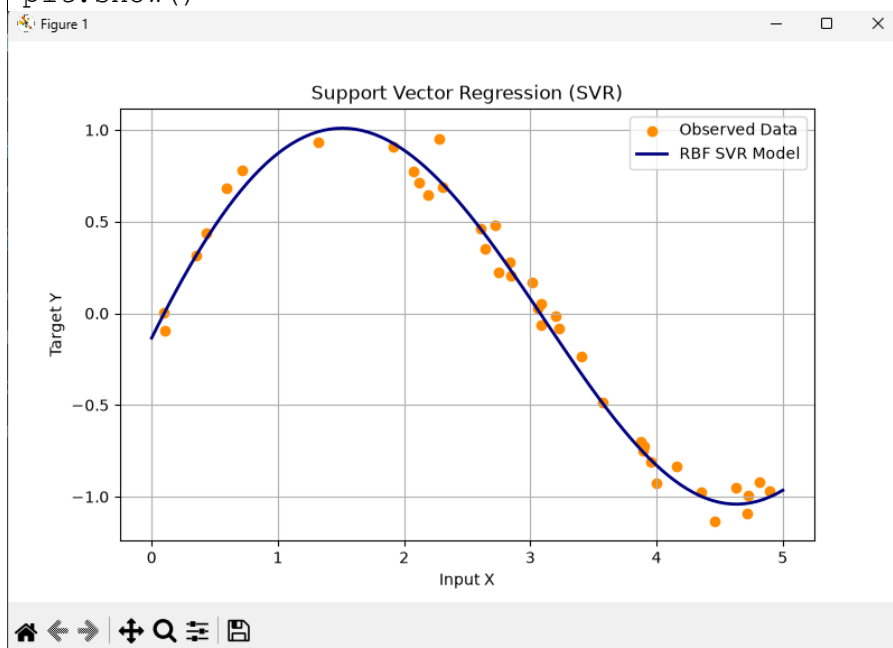
# 1. Generate non-linear sample data (Sine wave + noise)
np.random.seed(0)
X = np.sort(5 * np.random.rand(40, 1), axis=0)
y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0])

# 2. Fit SVR model with Radial Basis Function (RBF) kernel
svr_rbf = SVR(kernel='rbf', C=100, gamma=0.1, epsilon=0.1)
svr_rbf.fit(X, y)

```

```
# 3. Predict across continuous range for smooth curve
X_plot = np.linspace(0, 5, 100)[: , np.newaxis]
y_pred = svr_rbf.predict(X_plot)

# 4. Visualisation
plt.figure(figsize=(8, 5))
plt.scatter(X, y, color='darkorange', label='Observed Data')
plt.plot(X_plot, y_pred, color='navy', lw=2, label='RBF SVR
Model')
plt.xlabel("Input X")
plt.ylabel("Target Y")
plt.title("Support Vector Regression (SVR)")
plt.legend()
plt.grid(True)
plt.show()
```



Exercise 3.15

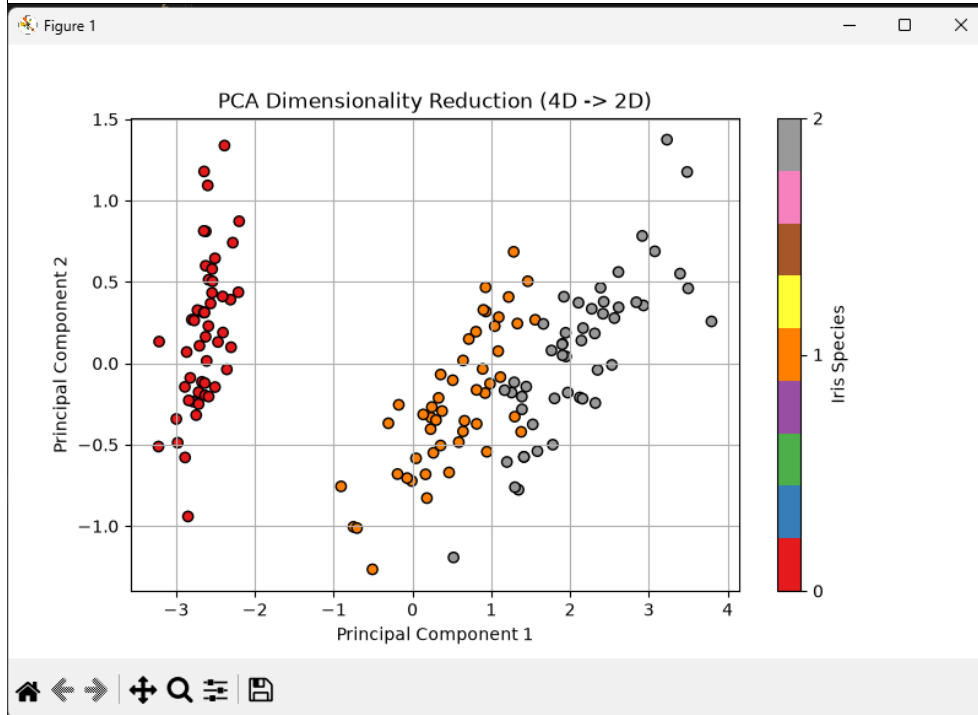
```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# 1. Load 4-dimensional Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# 2. Reduce features from 4D down to 2D using PCA
pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X)

# 3. Plot 2D projection
plt.figure(figsize=(8, 5))
scatter = plt.scatter(X_reduced[:, 0], X_reduced[:, 1], c=y,
cmap='Set1', edgecolors='k')
```

```
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Dimensionality Reduction (4D -> 2D)")
plt.colorbar(scatter, ticks=[0, 1, 2], label='Iris Species')
plt.grid(True)
plt.show()
```



Exercise 3.16

```
# Modified Q_test.py for an 8-State Routing Problem
import numpy as np
import matplotlib.pyplot as plt
from Q_Utils import *

# 1. Routing edges for states 0-7 (Goal = 7)
points_list = [(0, 1), (0, 2), (1, 3), (2, 4), (3, 5), (4, 5), (5, 6), (6, 7)]
goal = 7

# Display routing graph
showgraph(points_list)

# 2. Number of states
MATRIX_SIZE = 8

# Create R and Q matrices
R = createRmat(MATRIX_SIZE, points_list, goal)
Q = np.matrix(np.zeros([MATRIX_SIZE, MATRIX_SIZE]))
gamma = 0.8

# 3. Training Loop
scores = []
for i in range(700):
    current_state = np.random.randint(0, int(Q.shape[0]))
```

```

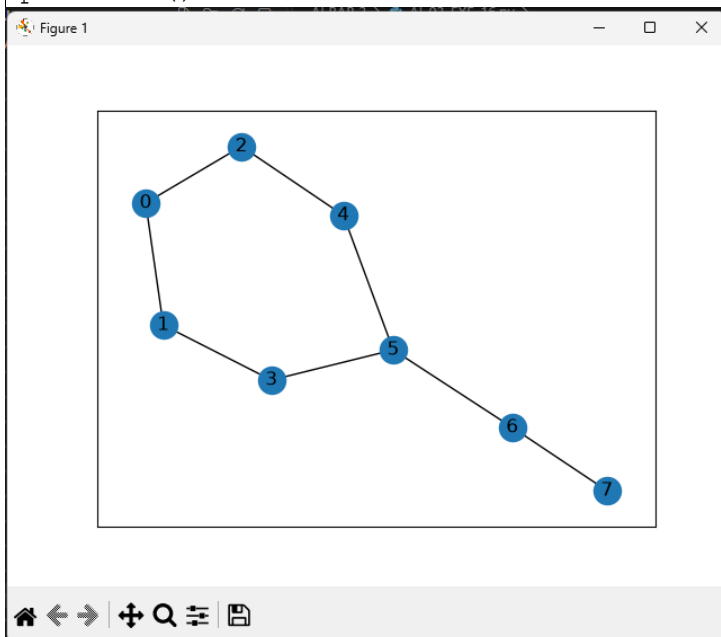
    available_act = available_actions(R, current_state)
    action = sample_next_action(available_act)
    score = update(R, Q, current_state, action, gamma)
    scores.append(score)

print("Trained Q matrix:")
print(Q / np.max(Q) * 100)

# 4. Testing Phase
current_state = 0
steps = [current_state]
while current_state != goal:
    next_step_index = np.where(Q[current_state,] ==
np.max(Q[current_state,]))[1]
    if next_step_index.shape[0] > 1:
        next_step_index = int(np.random.choice(next_step_index,
size=1))
    else:
        next_step_index = int(next_step_index)
    steps.append(next_step_index)
    current_state = next_step_index

print("Most efficient path:", steps)
plt.plot(scores)
plt.show()

```



Exercise 3.17

```

from sklearn import datasets
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier,
VotingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier

```

```

iris = datasets.load_iris()
X, y = iris.data[:, 1:3], iris.target

clf1 = LogisticRegression(random_state=1)
clf2 = RandomForestClassifier(n_estimators=50, random_state=1)
clf3 = GaussianNB()
clf4 = SVC()
clf5 = KNeighborsClassifier(n_neighbors=5) # Added K-Nearest
Neighbors

eclf = VotingClassifier(
    estimators=[('lr', clf1), ('rf', clf2), ('gnb', clf3), ('svc',
clf4), ('knn', clf5)],
    voting='hard'
)

classifiers = [clf1, clf2, clf3, clf4, clf5, eclf]
labels = ['Logistic Regression', 'Random Forest', 'Naive Bayes',
'SVM', 'KNN', 'Ensemble']

for clf, label in zip(classifiers, labels):
    scores = cross_val_score(clf, X, y, scoring='accuracy', cv=5)
    print("Accuracy: %0.2f (+/- %0.2f) [%s]" % (scores.mean(),
scores.std(), label))

```

```

Accuracy: 0.95 (+/- 0.04) [Logistic Regression]
Accuracy: 0.94 (+/- 0.04) [Random Forest]
Accuracy: 0.91 (+/- 0.04) [Naive Bayes]
Accuracy: 0.95 (+/- 0.04) [SVM]
Accuracy: 0.95 (+/- 0.04) [KNN]
Accuracy: 0.95 (+/- 0.04) [Ensemble]

```

Exercise 3.18

```

import pandas as pd
from flaml import AutoML
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split

# 1. Fetch California Housing data
housing = fetch_california_housing(as_frame=True)
df = housing.frame

# 2. Train-test split
df_train, df_test = train_test_split(df, test_size=0.2,
random_state=42)

# 3. Instantiate AutoML Predictor
automl = AutoML()

# Define settings for the training
settings = {
    "time_budget": 60,          # Maximum time (in seconds) to let it
search for models

```

```

    "metric": 'r2',          # The metric to optimize (R-squared
for regression)
    "task": 'regression',    # Type of machine learning problem
    "verbose": 0             # Set to 1 or 2 if you want to see the
progress in the console
}

# 4. Train the model
print("Training models...")
automl.fit(dataframe=df_train, label='MedHouseVal', **settings)

# 5. Evaluate the model
print(f"Best ML model found: {automl.best_estimator}")
print(f"R2 Score on test data: {automl.score(df_test,
df_test['MedHouseVal'])}")

```

```

Training models...
Best ML model found: lgbm
R2 Score on test data: 0.8398866274809992

```

Exercise 3.19

```

import pandas as pd
from sklearn.datasets import load_breast_cancer
from pycaret import classification

# 1. Load Breast Cancer dataset as DataFrame
cancer = load_breast_cancer(as_frame=True)
df = cancer.data
df['Target'] = cancer.target

# 2. Setup PyCaret Classification Pipeline
classification.setup(data=df, target='Target', session_id=123)

# 3. Evaluate and Compare Models
best_model = classification.compare_models()

```

Exercise 3.20

```

import lazypredict
from lazypredict.Supervised import LazyClassifier
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split

# 1. Load Wine dataset
X, y = load_wine(return_X_y=True)

# 2. Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# 3. Run LazyClassifier
clf = LazyClassifier(verbose=0, ignore_warnings=True,
custom_metric=None)

```

```
models, predictions = clf.fit(X_train, X_test, y_train, y_test)

print(models)
```

RandomForestClassifier	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.195728
LGSMClassifier	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.096486
NearestCentroid	0.977778	0.981481	1.000000	0.977985	0.979487	0.977778	0.025464
Perceptron	0.977778	0.981481	NaN	0.977819	0.979167	0.977778	0.021295
LinearSVC	0.977778	0.981481	NaN	0.977819	0.979167	0.977778	0.019983
LogisticRegression	0.977778	0.981481	1.000000	0.977985	0.979487	0.977778	0.029854
PassiveAggressiveClassifier	0.977778	0.981481	NaN	0.977819	0.979167	0.977778	0.021665
CalibratedClassifierCV	0.977778	0.981481	0.997531	0.977819	0.979167	0.977778	0.090897
SGDClassifier	0.977778	0.981481	NaN	0.977819	0.979167	0.977778	0.031964
RidgeClassifier	0.977778	0.981481	NaN	0.977819	0.979167	0.977778	0.031783
RidgeClassifierCV	0.977778	0.981481	NaN	0.977819	0.979167	0.977778	0.031330
NuSVC	0.977778	0.972222	NaN	0.977595	0.978947	0.977778	0.023306
SVC	0.977778	0.972222	NaN	0.977595	0.978947	0.977778	0.029767
QuadraticDiscriminantAnalysis	0.977778	0.972222	1.000000	0.977595	0.978947	0.977778	0.037201
BaggingClassifier	0.977778	0.972222	0.999327	0.977595	0.978947	0.977778	0.049224
AdaBoostClassifier	0.955556	0.962963	0.991395	0.955051	0.958654	0.955556	0.177182
KNeighborsClassifier	0.955556	0.962963	0.997613	0.955051	0.958654	0.955556	0.029010
BernoulliNB	0.955556	0.962963	0.995144	0.955051	0.958654	0.955556	0.028951
LabelSpreading	0.955556	0.962963	0.996857	0.955051	0.958654	0.955556	0.025172
LabelPropagation	0.955556	0.962963	0.993154	0.955051	0.958654	0.955556	0.025113
DecisionTreeClassifier	0.955556	0.950000	0.964815	0.955373	0.956725	0.955556	0.022046
XGBClassifier	0.933333	0.935185	0.997007	0.933287	0.937255	0.933333	1.142129
ExtraTreeClassifier	0.888889	0.881481	0.911111	0.889208	0.895556	0.888889	0.022919
DummyClassifier	0.400000	0.333333	0.500000	0.228571	0.160000	0.400000	0.022001

Exercise 3.21

```
import lazypredict
from lazypredict.Supervised import LazyRegressor
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split

# 1. Load Diabetes dataset
X, y = load_diabetes(return_X_y=True)

# 2. Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Run LazyRegressor
reg = LazyRegressor(verbose=0, ignore_warnings=True,
                    custom_metric=None)
models, predictions = reg.fit(X_train, X_test, y_train, y_test)

print(models)
```

GradientBoostingRegressor	0.385235	0.455095	53.730733	0.171533
ElasticNet	0.384870	0.454771	53.746672	0.015054
RidgeCV	0.384165	0.454147	53.777454	0.029693
Ridge	0.384165	0.454147	53.777454	0.017877
TransformedTargetRegressor	0.382424	0.452603	53.853446	0.018547
LinearRegression	0.382424	0.452603	53.853446	0.015579
HuberRegressor	0.375897	0.446818	54.137276	0.028126
AdaBoostRegressor	0.370349	0.441900	54.377352	0.202483
RandomForestRegressor	0.369863	0.441469	54.398351	0.403425
TweedieRegressor	0.358690	0.431566	54.878488	0.024474
GammaRegressor	0.352424	0.426012	55.145951	0.023170
KNeighborsRegressor	0.351066	0.424809	55.203713	0.018811
PassiveAggressiveRegressor	0.339659	0.414698	55.686813	0.014774
LGBMRegressor	0.313133	0.391186	56.794270	0.088021
HistGradientBoostingRegressor	0.307952	0.386594	57.008045	0.166490
BaggingRegressor	0.298547	0.378258	57.394119	0.055768
XGBRegressor	0.286427	0.367515	57.887837	1.321176
LinearSVR	0.182492	0.275390	61.960412	0.014997
OrthogonalMatchingPursuit	0.135062	0.233350	63.732456	0.015189
SVR	0.077373	0.182217	65.823540	0.030598
NuSVR	0.061207	0.167888	66.397714	0.023487
DecisionTreeRegressor	-0.040653	0.077603	69.907081	0.016912
QuantileRegressor	-0.131240	-0.002690	72.886244	0.141502
DummyRegressor	-0.141702	-0.011963	73.222493	0.013209
RANSACRegressor	-0.222826	-0.083869	75.779298	0.226977
ExtraTreeRegressor	-0.262973	-0.119453	77.013205	0.014757
GaussianProcessRegressor	-0.804769	-0.599682	92.061693	0.083411
MLPRegressor	-1.131768	-0.889522	100.054846	0.355502
Lars	-1.486155	-1.203638	108.051873	0.021623
KernelRidge	-4.907095	-4.235834	166.554027	0.053835